

Caso: Plataforma de Gestión de Pólizas

Nuestro negocio maneja dos tipos de pólizas para arrendamiento de inmuebles: Individual y Colectiva. Las pólizas colectivas están orientadas a las inmobiliarias y Administraciones de copropiedades, se aseguran a los arrendatarios de los inmuebles y en caso de siniestros los beneficiarios son los arrendadores. En las pólizas individuales el tomador y el asegurado es el arrendatario y el beneficiario es el arrendador.

Una póliza colectiva puede tener uno o muchos riesgos, en una póliza individual solo tiene un riesgo.

Todas las pólizas cuentan con: periodo de vigencia, un valor de canón mensual de arrendamiento, un valor de prima que corresponde al valor canon mensual por número de meses de la vigencia, igualmente, todas las pólizas pueden ser renovadas por el mismo periodo de vigencia inicial y para ello se debe ajustar el valor de canon según incremento del IPC.

El negocio necesita una plataforma que permita:

1. Crear, consultar y modificar pólizas (individuales y Colectivas).
2. Renovación automática basada en reglas de negocio.
3. Eventos de notificación (correo/SMS) para la creación y renovación.
4. Integración con un CORE transaccional legado.
5. Disponibilidad 24/7 y resiliencia.

Se requiere implementar un API que permita la gestión de pólizas así:

1. Para colectivas: Agregar riesgos, eliminar (cancelar) riesgos
2. Para individuales: Soportar los consumos necesarios del front.
3. Todas las acciones que modifiquen estados de pólizas y/o riesgos, deben consumir un servicio agnóstico de edición, disponibilizado a través de capa media en weblogic, el cuál mantiene actualizado el sistema CORE de seguros.

Actividades a realizar

1. Diseñar la arquitectura de alto nivel del sistema.

Componentes Principales:

- **API Gateway:** en este punto de entrada único para el front end, se necesita enrutar las pólizas individuales y colectivas a los microservicios correspondientes por tanto se hace importante el uso de verificación, esto haciendo uso del marcado xml, JSON o otras especies de tecnologías, por lo cual podría ser uso del mecanismo de autenticación de tipo SHA512 o cualquier otro para validar la información de punto a punto.
- **Microservicio de Gestión de Pólizas:** Al manejo y la administración del CRUD se necesita implementar la lógica del negocio para gestionar las pólizas individuales, teniendo un solo riesgo, y las colectivas manejar un arreglo dinámico de tipo stack (o pila con con prioridad de la de mayor valor, mayor importancia, o mayor relevancia a la menor prioridad, importancia o relevancia)
- **Microservicio de Renovaciones (Job Worker):** Aunque pongo en duda este sistema, ya que en caso de caerse, podría generarse problemas a todos nuestros sistemas, esto debido a la gran cantidad de usuarios, gestiones, y sistemas informáticos, que de él dependen, (caso Bancolombia), Este sería un sistema aislado que se periódicamente (ej. cada noche, 6 días a la semana), Buscando pólizas que se encuentran próximas a vencer, calcula el nuevo canon (+ IPC anual) y genera la renovación, Además de esto también es importante en cuenta las novedades de cada póliza la individual o colectivo.
- **Bus de Eventos:** este es el corazón de la arquitectura, todo lo que nosotros trabajemos de acuerdo a servicios, transacciones, procesos, y demás debemos validar en cualquier sistema de bus de eventos sea Kafka o RabbitMQ o como un Message Broker relevance (MBR),. y en caso de fallo la revisión debería sacarse de aquí.
- **Microservicio de Notificaciones:** Nosotros necesitamos dos tipos de información, el primero del sistema de verificación o bus de eventos nuestra base de datos a nosotros como propietarios, el segundo de la base de datos a nuestros clientes previo verificación por medio de correo electrónico, SMS, o comunicación directa en estos sistemas de encolaje llega a fallar, sin afectar la creación de nuevas pólizas.
- **Microservicio de Integración:** Capa Anticorrupción - ACL, este servicio escucha los eventos que cambia de estados o riesgos, y transforma el modelo de datos a la plataforma median te el sistema de WebLogic.

- **Resiliencia total:** si WebLogic se cae por mantenimiento, el bus de eventos guarda los mensajes (queue retrieval system), tu sistema sigue operando 24/7 creando pólizas en tu base de datos local y, cuando WebLogic regrese, la Capa Anticorrupción procesará la cola atrasada.

Escalabilidad: Podría escalar solo el servicio de notificaciones en épocas de renovaciones masivas sin tocar el resto, O el sistema de Api GateWay, el microservicio de gestion de polizas, o el microsistema de renovación que seria el más delicado.

Componentes Principales:

- **Web API:** Una sola aplicación grande que contiene todos los módulos: Gestión de Pólizas, Renovaciones y Notificaciones, compartiendo los mismos recursos de memoria

y procesamiento, por lo se necesita de un bus de eventos como kafka que valide punto a punto toda la trazabilidad de este proceso.

- **Llamadas Síncronas:** Cuando el Front-end pide crear una póliza y modificar un riesgo, el sistema:
 1. Validar la información, que NO sea repetida, errónea, falta, y se puede validar a los distintos entes de control (policia, fiscalia, o entes de control como es la base de datos de microcredito, o demás empresas).
 2. Validado este punto, vamos a la comprobación en el sistema de datos de nosotros.
 3. Después vamos a Guarda en su base de datos.
 4. *Espera* a que el proveedor de SMS responda.
 5. *Espera* a que la capa media en WebLogic (CORE) responda exitosamente.
 6. Recién ahí, le responde "OK" al Front-end.
- **Job Scheduler Interno:** Las renovaciones automáticas corren en la misma aplicación, compitiendo por CPU con los usuarios que están intentando crear pólizas.

2. Seleccionar

3 patrones de arquitectura y justificar por qué: ○ Ej.: event-driven, hexagonal, CQRS, API Gateway, microservicios, etc.

los patrones de arquitectura, pueden ser variados, bien o mal estructurados por lo que ha decido ver solo los más relevantes y tambien los no relevantes, por qué eso no depende del patrón de arquitectura si no de la implementación de los mismos, por lo que es decir o al ser implementado mejor o no se a implementado mejor, queda más en los errores conceptuales del equipo de trabajo, por ejemplo: un patrón de arquitectura como es CQRS, bien estructurado, bien implementado, desempeña igual o mejor que un patrón de arquitectura hexagonal que haya quedado mal implementando, eso depende de la madurez y experiencia del arquitecto, los implementadores, las personas que están alrededor, el tráfico y calidad de los datos, el tráfico y calidad de los errores, y el how do (la herramienta, el proceso, y las expectativas al implementarlo).

Event Driven

- Las modificaciones de este tipo de venir al CORE en WebLogic, y lo hace sincrónico o asincrónico nuestro sistema debe obtener la información relevante al dato cargado, por ejemplo: en nuestro proceso anterior, tenemos que ver si la información proviene los datos Individual o Colectivos, masivos o en bloque, serial o paralelo para obtencion de datos, entonces el event driven depende de las entradas, el procesamiento de los datos y la salida, por que sería de vital importancia el uso de broker de mensajería y el bus de eventos (sea cual sea Kafka o RabbitMQ), en cada proceso dejar un marcado especial de la traza:

Por ejemplo: API Gateway aparezca y metodo de abre proceso API, procesando y saliendo API, después gestión de polizas de igual manera, bus de eventos,

notificaciones, integración y demás esto para hacer más fácil todo el proceso de información y para llevar la trazabilidad de cada póliza.

- importante los correos y SMS por creación o renovación no deben bloquear el flujo principal. EDA permite que el servicio de notificaciones simplemente "escuche" los eventos de creación/renovación y actúe de forma independiente esto con el objetivo de desacoplar las notificaciones.

Arquitectura Hexagonal

- **Protección del Dominio (Reglas de Negocio):** al tener reglas claras: los cálculos de primas (canon mensual * meses), la diferenciación de riesgos entre pólizas individuales y colectivas, y el cálculo del incremento del IPC para renovaciones, uno de los motivos por lo que la arquitectura hexagonal se asegura que este código puro de negocio no se contamine con librerías de bases de datos o librerías XML/SOAP antiguas que pueda requerir WebLogic Applications, separación de las mismas a cada microservicio, servicio, aplicabilidad y contenciones a posibles fallos.
- **Facilita las Pruebas:** la lógica de pólizas aislada en el "centro" del hexágono, puedes correr pruebas unitarias automáticas sobre el cálculo de las renovaciones y las reglas de riesgos sin necesidad de tener la base de datos o WebLogic App levantados, cada microservicio podría verse como un sistema aparte y poder controlar como si fuera una caja de fusibles, si falla uno el circuito continúa y los datos serían la electricidad de ese circuito.

API Gateway

- **Soporte optimizado para el Front-end:** al tener el API Gateway podemos usarlo como materia fuente de muchas más aplicaciones debido a que podemos tener un API cifrado para nuestras transacciones, y un API abierto pero vigilado para nuestros terceros quienes revisan y adoptan las medidas necesarias para la implementación, de modo que este sistema quede cubierto hacia futuros desarrollos de software, futuros cambios de requerimientos o la latencia en los servicios, se exigen "soportar los consumos necesarios del front", El front-end no debería saber si internamente tienes un microservicio para pólizas individuales y otro para colectivas, o si las renovaciones están en otro lado. El API Gateway enruta la petición al lugar correcto de forma transparente, conociendo el trabajo del mismo y ofreciendo a nuestros socios, terceros o partners una estructura másiva de servicios a utilizar por ejemplo: si un cliente decide que el necesita consultar la información de una póliza colectiva y conocer los que desean cambiar, modificar y agregar podríamos ofrecerle a este tercero, un API Rest con los datos fiables y cobrarle por un consumo de dicha operación o por millon de operaciones CTO (Cost per thousand Operations).
- **Transformación y Agregación:** Para demostrar una póliza colectiva el front-end necesita datos de la póliza y datos del cliente, el API Gateway puede hacer ambas llamadas a los microservicios internos y devolverle al front-end un solo objeto JSON

consolidado, reduciendo la latencia de red, un favor dos mandados d'ría yo por qué esta arquitectura es costosa con referencia al uso de la información, mientras más mejor o sea si podemos cargar un Api Rest con la carga de todos los datos podemos utilizar de una mejor manera la carga de datos ofreciendo flexibilidad.

- **Seguridad y Control:** sería un lugar ideal para centralizar la autenticación (tokens JWT) y limitar la cantidad de peticiones (Rate Limiting) para proteger tu plataforma o nuestros distintos sistemas de información relevantes, para poder controlar nuestro sistema de entrada, procesos y salida.

Y por que **NO** escoger los otros modelos de de arquitectura:

Monolito en Capas (N-Tier Monolithic Architecture)

- **Por qué NO escogerlo para este caso:**
 - **Peligro para la disponibilidad 24/7:** en un caso hipotético, si el proceso de "Renovación automática" corre a fin de mes e incrementa el IPC de miles de pólizas simultáneamente, consumirá toda la CPU y memoria de la aplicación. Esto provocará que el Front-end (que vive en el mismo servidor) se vuelva inoperante para los usuarios que intentan crear una póliza nueva, por lo que sería conveniente pensar en la sepacion de poder utilizar esta arquitectura, y más si es bien sabido que era el antiguo sistema que se usaba por lo cual debe ser muy delicado entrar a reparar el problema, error o deficiencia a cada uno, el problema de los software legados como sería un AS400, Cobol o demás tecnologia a lo cual iria de la mano del siguiente caso de estudio, el despliegue riesgoso.
 - **Despliegues riesgosos:** solo necesitas modificar el texto del correo electrónico de "Notificación de creación", tendrías que re-desplegar toda la plataforma completa,. si algo sale mal, tumbas el módulo de pólizas y la integración con el CORE legado lo cual traeria problemas de uso, por lo que KISS podria ser la mejor opción, (Kept it Simple, Stupid).
 - **Falta de resiliencia:** Un error de memoria (Out of Memory) en la conexión con WebLogic apagará todo el sistema crítico, una operacion podria consumir recursos pero 1000 o un millon de transacciones de un instante, aunque eso tambien estaria en el uso de cualquier sistema critico constantemente para evitar una denegacion de servicios por una infiltracion, si tenemos eterna vigilancia de este sistema podriamos solventar muchos casos.

Microservicios con Comunicación Síncrona:

- **Por qué NO escogerlo para este caso:**

- **Fallo en Cascada (Cascading Failures):** dentro de los requerimientos, el número 3 exigen que todas las acciones de modificación se consuman un servicio en WebLogic, si usas comunicación síncrona o hasta asíncrona, el API llamará a WebLogic y se quedará esperando (time out, time wait, time await, time out y hasta el time offline las 5 Time de espera), si WebLogic está caído por mantenimiento o la red está lenta, tu petición dará un *Timeout*, el usuario no podrá guardar la póliza en el Front-end, rompiendo tu promesa de disponibilidad 24/7.
- **Acoplamiento y desplomamiento temporal:** esto obliga a que todos los sistemas (el proveedor de SMS/correos, la base de datos y el CORE) estén 100% operativos exactamente en el mismo milisegundo en que el usuario presiona "Guardar póliza", eso es matemáticamente insostenible a largo plazo.
- **HTTP/REST:** Este patrón ocurre cuando los equipos adoptan microservicios, pero los conectan entre sí mediante llamadas directas HTTP/REST (punto a punto) esperando la respuesta inmediata de cada uno, la solución sería asíncrono mientras espera

CQRS (Command Query Responsibility Segregation)

Por qué NO escogerlo para este caso:

- **Over-engineering:** CQRS es brillante para sistemas con una disparidad brutal entre lecturas y escrituras por ejemplo, Twitter o un e-commerce en Black Friday, pero para un banco que utiliza las pólizas de arrendamiento, el volumen transaccional de "crear, consultar y modificar" es relativamente predecible y balanceado, por lo cual sería descartado en el proceso de (IN - OUT) no en el proceso.
- **Complejidad innecesaria de Consistencia Eventual:** al separar la base de datos de lectura de la de escritura (error fundamental), cuando un usuario cree un riesgo en una póliza colectiva, tendrá que sincronizar ambas bases, dicho usuario podría recargar la pantalla y no ver el riesgo que acaba de crear porque la base de datos de "lectura" aún no se ha actualizado, esto genera confusión en plataformas financieras/aseguradoras donde la precisión inmediata en la interfaz es clave, para un CRUD avanzado con reglas de IPC, CQRS aporta más problemas que beneficios.
- **CQRS** es un patrón moderno muy famoso que separa las operaciones de lectura (Consultas/Queries) de las de escritura (Comandos/Commands), pero genera controversia debido a que se utilizan modelos de datos e incluso bases de datos físicamente distintas para cada una.

Entonces como digo, el problema no es, si se desea una o otra tecnología, uno u otro patrón sino la implementación del mismo, la experiencia y madurez del equipo de trabajo, que en una empresa funciona y en otra no, no podría salir del éxito del procedimiento viene directamente a las personas que quieren lo implementaron.

3. Describir el modelo de datos principal (sin entrar en SQL detallado).

classDiagram

```
class Poliza {  
    <<abstract>>  
    +String idPoliza  
    +Date fechaInicioVigencia  
    +Date fechaFinVigencia  
    +Decimal canonMensual  
    +Decimal valorPrima  
    +String estado  
    +calcularPrima()  
    +renovar(Decimal incrementoIPC)  
}
```

```
class PolizaIndividual {  
    +String idTomadorAsegurado  
}
```

```
class PolizaColectiva {  
    +String idTomadorInmobiliaria  
    +agregarRiesgo(Riesgo r)  
    +eliminarRiesgo(String idRiesgo)  
}
```

```
class Riesgo {  
    +String idRiesgo  
    +String direccionInmueble  
    +String idAseguradoArrendatario  
    +String idBeneficiarioArrendador  
    +String estado  
}
```

```
class Tercero {  
    +String idTercero  
    +String tipoDocumento  
    +String numeroDocumento  
    +String nombreCompleto  
    +String emailContacto  
}
```

Poliza <|-- PolizaIndividual : Hereda de

Poliza <|-- PolizaColectiva : Hereda de

PolizaIndividual "1" *-- "1" Riesgo : Tiene (Composición)

PolizaColectiva "1" *-- "1..*" Riesgo : Tiene (Composición)

Riesgo --> Tercero : Referencia a

Poliza --> Tercero : Referencia a

integración con el CORE legado. Aquí el detalle de cada bloque:

Un breve explicación de lo que se desea, dar conocer

1. Clase Base: Poliza (Abstracta)

Qué hace: Centraliza todo lo que es común a cualquier póliza según tus reglas, no puedes instanciar una "Póliza" genérica; siempre debe ser Individual o Colectiva.

Atributos clave: Periodo de vigencia (fechaInicio, fechaFin), el canonMensual, y el valorPrima.

Lógica embebida: Tiene los métodos (o la responsabilidad en el microservicio) para calcularPrima() (canon x meses) y renovar(), esto lo cual garantiza que la regla del IPC aplique de forma estándar para todas.

2. Las Clases Hijas: PolizaIndividual y PolizaColectiva (separación de las clases) tomando como base el objeto Póliza.

Herencia: Ambas heredan los atributos de la clase base Poliza.

Diferenciación del Tomador: En la Individual, el tomador es el mismo arrendatario (apuntamos a un ID de Tercero). En la Colectiva, el tomador es la Inmobiliaria/Administración.

Restricción de Riesgos (El Core del Negocio):

La **PolizaIndividual** tiene una relación de composición 1 a 1 con la clase Riesgo. Solo puede existir un riesgo por póliza, individual tendrían formatos común y para todos .

La **PolizaColectiva** tiene una relación de composición 1 a muchos (1..*) con la clase Riesgo. Además, expone los métodos específicos requeridos en tu API: AgregarRiesgo() y eliminarRiesgo() colectiva vendría que un formato específico, para cada uno de nuestros clientes, por lo que esta clase tendría preponderancia sobre las pólizas individuales.

3. Clase Riesgo

Qué hace: Representa un inmueble asegurado y las partes involucradas directamente en ese inmueble, Los valores y costos tendrían siendo tomados las pólizas individuales y colectivas.

Atributos clave: Dirección del inmueble, quién es el Asegurado (Arrendatario) y quién es el Beneficiario (Arrendador).

Por qué separarlo: Al separar el "Riesgo" de la "Póliza", esto permite que una póliza colectiva englobe a 50 arrendatarios distintos en 50 inmuebles distintos, cada uno con su propio beneficiario (arrendador), bajo un mismo paraguas (la póliza de la inmobiliaria).

4. Clase Tercero (Entidad de Referencia)

Qué hace: en un catálogo o tabla maestra de personas y empresas, en lugar de guardar el nombre y correo del arrendatario repetidas veces, guardamos su idTercero, esto es vital para el requerimiento de "Notificaciones (correo/SMS)", ya que los datos de contacto viven aquí de forma centralizada.

4. Explicar cómo manejaría:

○ Escalabilidad

- **Cómo:** Escalabilidad horizontal e independiente (gracias a los microservicios),
- **En la práctica:** Si a fin de mes el sistema colapsa por las *renovaciones automáticas masivas*, solo agregas más instancias (servidores/contenedores) al microservicio de **Renovaciones** y al de **Notificaciones**. El microservicio que atiende las peticiones del Front-end no se toca, ahorrando recursos y garantizando que los usuarios sigan navegando rápido, un nuevo contenedor de cualquier microservicio, el trabajo lo asume la nube mientras que los datos siguen funcionando, y el colocar alertas, por ejemplo 75%, 80% y 90% a modo de semáforo, jamás permitir que llegue a un 100% para el despliegue de cada contenedor por lo cual en 80% sería bueno para desplegarlo y si llega a fallar alertar al sistema para que el pueda tolerar la masa crítica mientras que se despliega el plan B de un nuevo instancia de servidor o contenedor (tolerancia al error).

○ Logs y observabilidad

- **Cómo:** Trazabilidad distribuida y logs centralizados (leer todo lo que el software nos dice) o sea tener pendiente que el software este respondiente a cualquier notificación del Bus de Eventos.
- **En la práctica:** Inyectar un **Correlation ID** (un código único) desde el API Gateway a cada petición del Front-end, ese código viaja por el microservicio, el evento y la capa anticorrupción. Todos los servicios envían sus logs a un punto central (ej. Elasticsearch/Kibana o Datadog), si una póliza falla en WebLogic, buscas ese ID y ves exactamente la ruta completa y en qué milisegundo se rompió., de hecho los microservicios hacen uso de un sistema de notificaciones al leer los logs llamado ILogs que se puede revisar en tiempo real.

○ Tolerancia a fallos

- **Cómo:** Circuit Breaker (Cortocircuito) y Colas de Mensajes Muertos (DLQ).
- **En la práctica:** Si el CORE legado en WebLogic se cae, la capa anticorrupción se activa el **Circuit Breaker**: esto deja de enviarle peticiones para no saturarlo, los eventos de las nuevas pólizas se guardan de forma segura en una cola, cuando WebLogic revive, el sistema reintenta enviarlos automáticamente, esto para el usuario final del Front-end, el sistema "nunca se cayó".

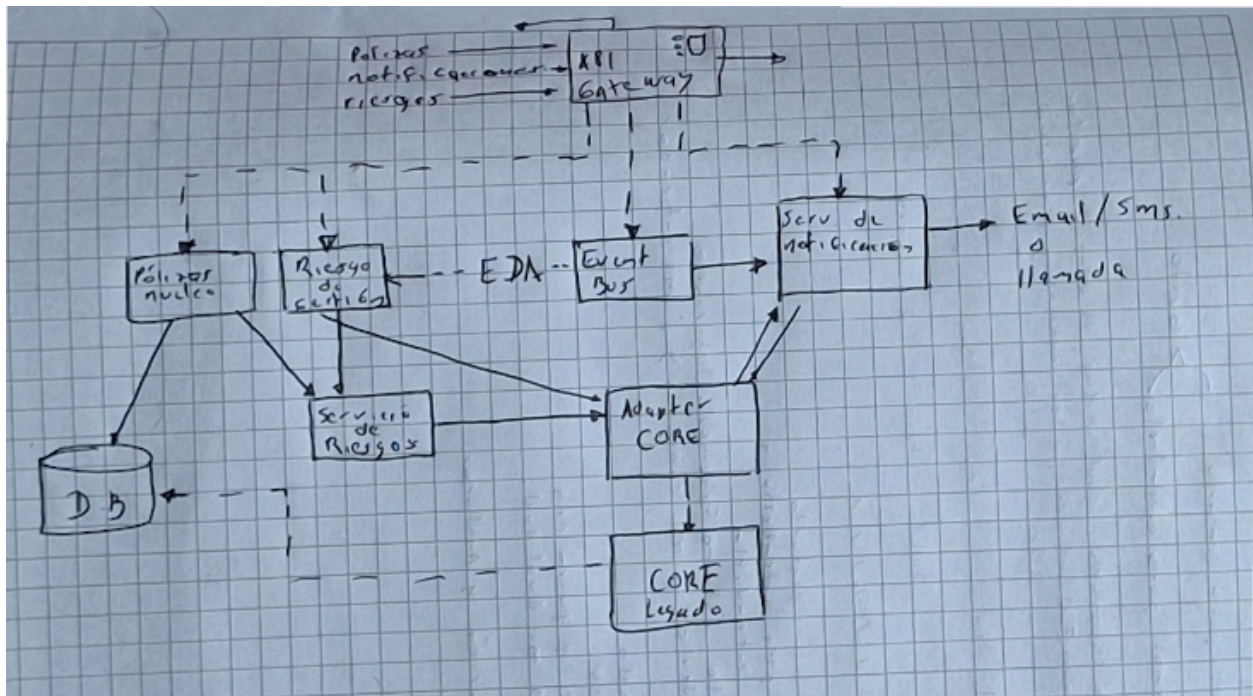
○ Versionamiento de APIs

- **Cómo:** Control centralizado a través del API Gateway.
- **En la práctica:** Las URLs del API Gateway llevarán la versión explícita (ej. /api/v1/polizas-colectivas)., si en un año el negocio pide cambiar drásticamente cómo se agregan los riesgos, creas un /api/v2/.... En nuestro Front-end actual sigue usando la **v1**

sin enterarse ni romperse, hasta que el equipo de Front end tenga tiempo de migrar a la nueva versión.

5. Diagramar (texto o dibujo simple) los componentes principales:

- Servicio de Pólizas
- Servicio de Riesgos
- Servicio de Notificaciones
- Adapter de integración con CORE
- Base de datos
- API Gateway



UML component diagram nada más una diagrama

Es una arquitectura de pólizas management platform, especificado el cuerpo de entto póliza servicio us. Representantor renete metrita. Servicio sonda me servicios y servicio notificaciones y manejar riesgos. Servicio de Notegona, interntent abletivos:

- pólizas
- /riesgos
- /riesgos
- /notificaciones

Outputs a dependencia recones

Roles:

- Servicio de Pólizas
- Gestio de Riesgos
- Resitos etc
- Notificaciones etc
- Appiok o Terceros

